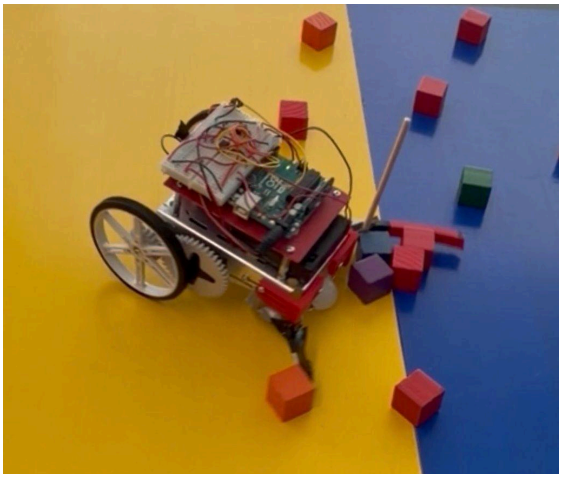
Team 35
Toby Huynh (thh45), Alexander Barry (ajb482), Nicole Kim (njk79)

1. Robot Design and Strategy Overview



Our robot had a gear system and an arm connected to a servo. The arm was connected to a servo to allow it to come down at the start of the round, while still fitting within the 8x8in starting area. The gear system had a 1:2 gear ratio which allowed our robot to move faster.

Ideally, the robot would drop its arm down and drive forward to reach the middle faster than the opposing robot. Then after reaching the middle, it would turn right by 90 degrees and push the majority of the blocks to the side where the other robot wouldn't be able to access them.

2. Design Process Reflection

We completed the mobility and color detection milestone fairly quickly and had a clear sense of how we wanted our robot to operate during the competition. Our initial design featured a much longer arm and smaller wheels. When we tested this configuration on the actual game arena, our robot was much slower than we had anticipated. Although our current design allowed the arm to wipe the board of nearly all the blocks, this was unrealistic in an actual match, particularly against opponents who had faster robots. To address this concern, we designed a gear system and installed larger wheels to allow us to get to the center of the arena much faster. However, due to size constraints we had to sacrifice some of the sweeping arm's length to compensate for the larger wheels and gear assembly.

3. Competition Analysis

We did very well during the tournament, with a final score of 5 wins, 0 losses, and 2 ties after the first round. Our design worked as expected for the majority of the rounds, and the two ties were due to a collision and to unlucky placement of the cubes. Unfortunately, in the first round of the final bracket, we forgot to turn on the 6V battery pack, causing us to lose the match. We didn't expect our robot to be as fast as it was, as in some rounds it was able to collect 12 cubes and park fully on the side before the other robot even touched a single cube. The strength of our robot was the speed. The main weakness was that if we missed enough blocks on the initial pass, then it'd just sit at the side of the board and wouldn't continue collecting more, allowing the opposing robot to surpass it (which never ended up happening during the competition).

Group 5					
Rank	Team	W-L-T	Ranking Points	Diff	PF
1	Team 35	5-0-2	17	29	46
2	Team 17	5-1-1	16	16	28
3	Team 37	4-1-2	14	4	28
4	Team 29	3-3-1	10	-4	18
5	Team 2	2-4-1	7	-5	19
6	Team 45	2-4-1	7	-12	18
7	Team 12	1-5-1	4	-14	8
8	Team 34	0-4-3	3	-14	16

First Round Bracket Standings

4. Conclusions

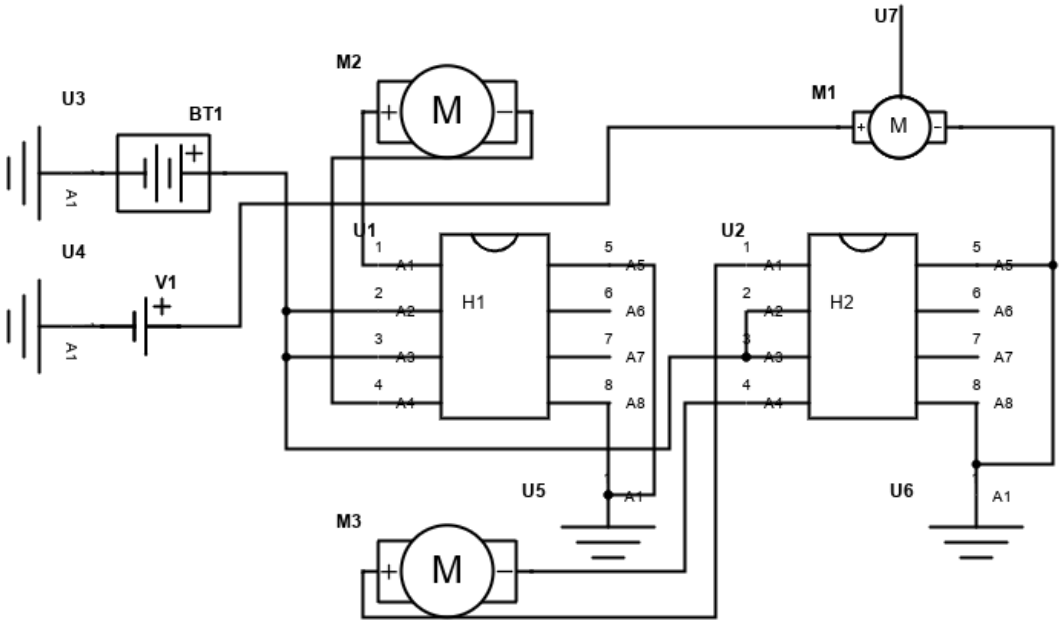
We would have designed the gear system differently. Our design resulted in the axis of the wheels being moved backward, which took away from how long the arm was able to extend to stay within the 12” diameter limit. A longer arm would have been a lot more effective.

5. Appendix A: bill of materials (BOM)

Item	Volume (in^3)	Weight (g)	Cost ($)
Left arm (3D print)	1.224	18.819	8.53
Right arm (3D print)	0.765	11.761875	5.70
Left mount (3D print)	0.405	6.226875	3.49
Right mount (3D print)	0.405	6.226875	3.49
Gears (3D print)	1.492	22.9395	10.18
Large wheels	n/a	n/a	4.00
Washers	n/a	n/a	1.50
Total			**36.89**

*A density of 20.5 g/in^3 for PLA was used to calculate the weight of 3D prints

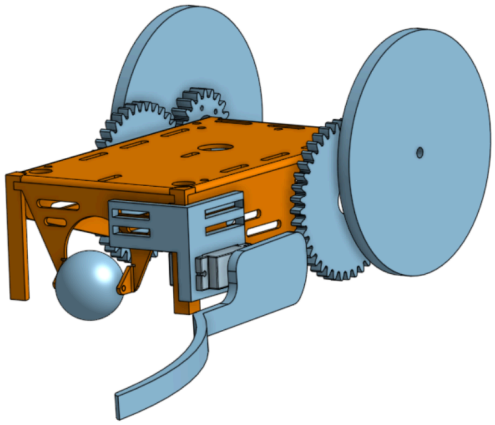
Appendix B: Circuit Diagram Schematic

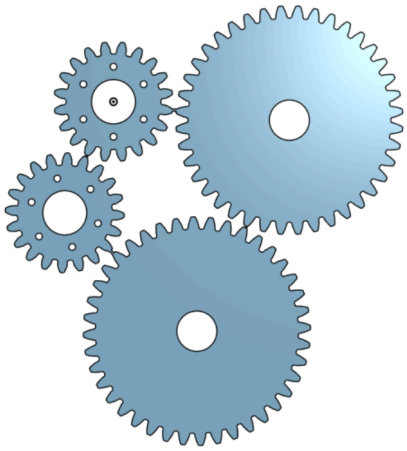


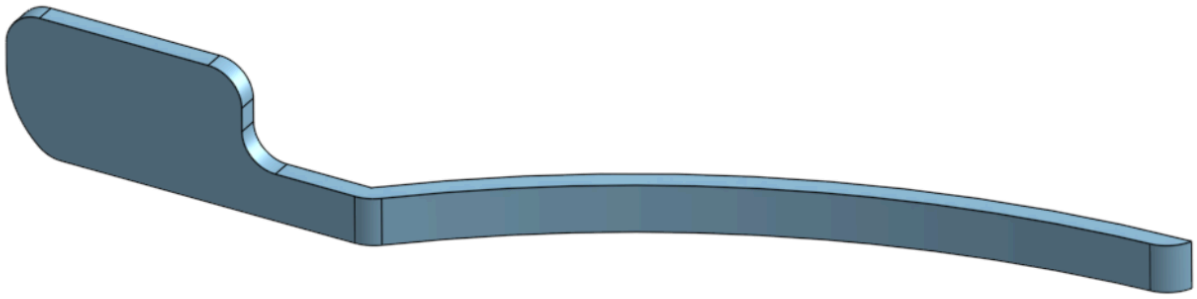
Notes:

M2 and M3 are the left and right wheel servo motors respectively.
M1 is the arm servo. Wire U7 connects to the Arduino Uno.
H1 and H2 Pins A6 and A7 connect to the Arduino Uno to receive wheel inputs.
BT3 is the 1.5V battery pack, V1 is the 5V supply from the Arduino Uno.

7. Appendix C: CAD files and drawings. Include screenshots of relevant CAD files or drawings used for manufacturing individual parts. Also include screenshots of part volumes and weight calculations (or photos of the parts being weighed) corresponding to the lines in your BOM.

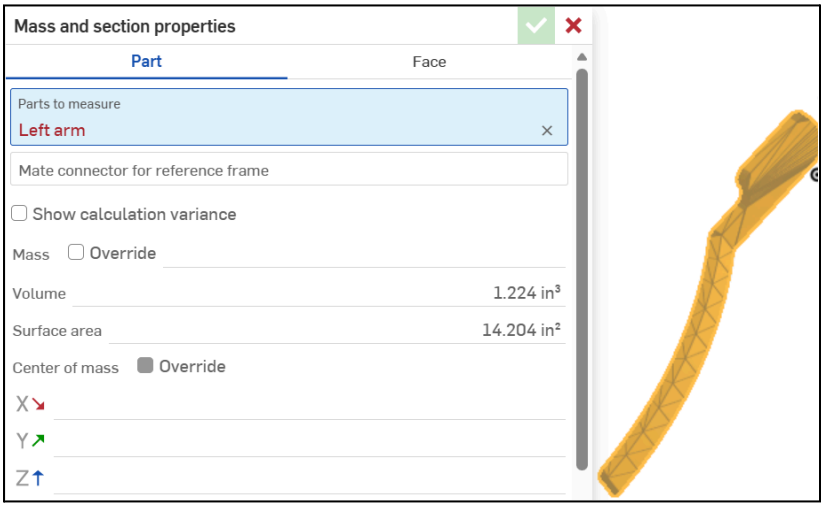




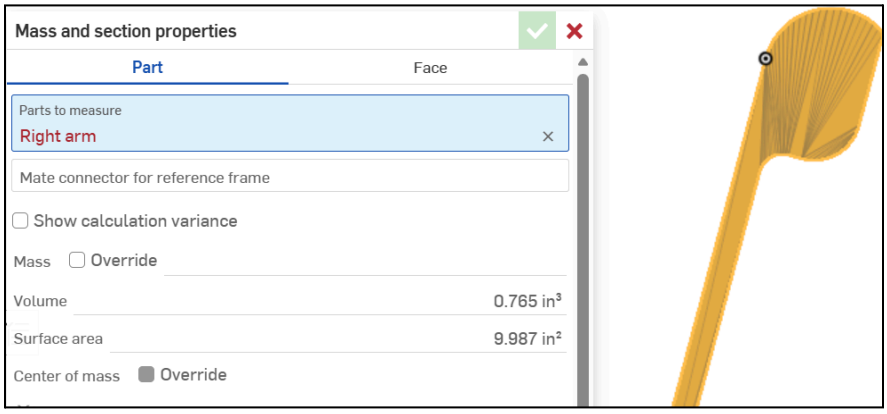


<u>Volume screenshots for 3d prints:</u>

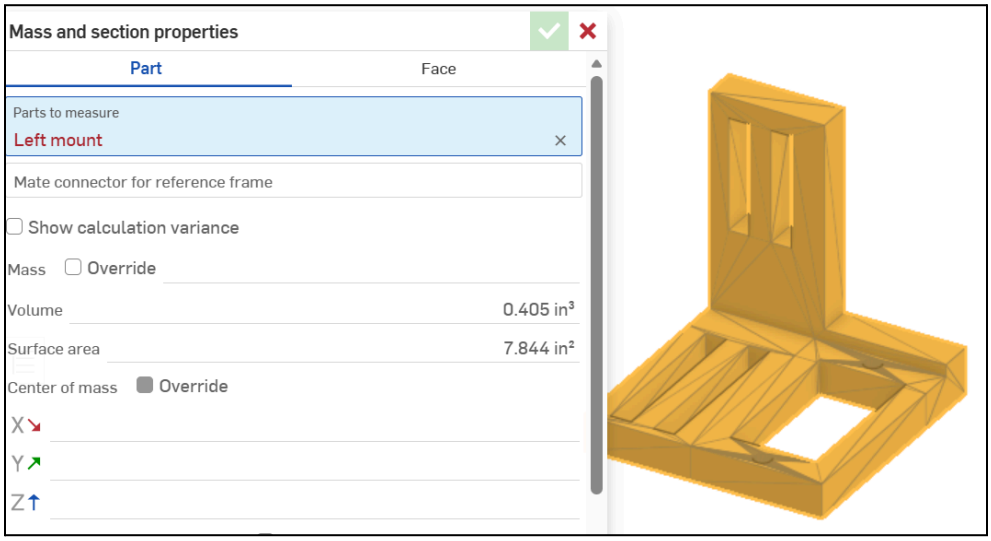
Left Arm:



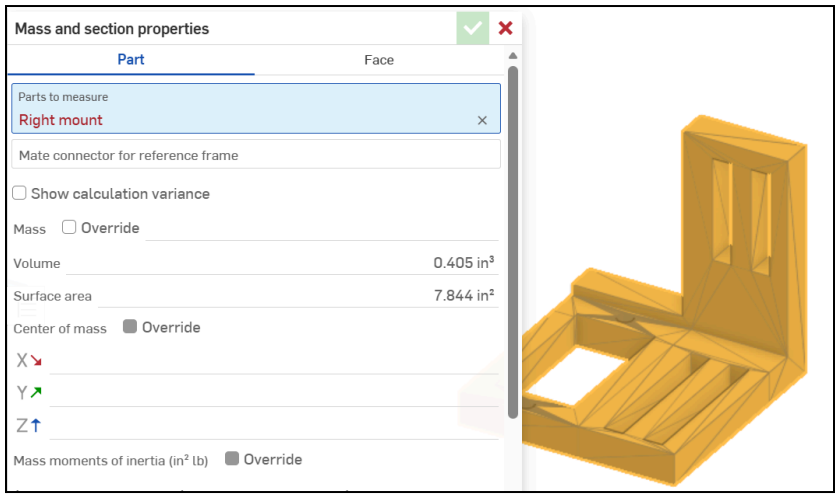
Right Arm:



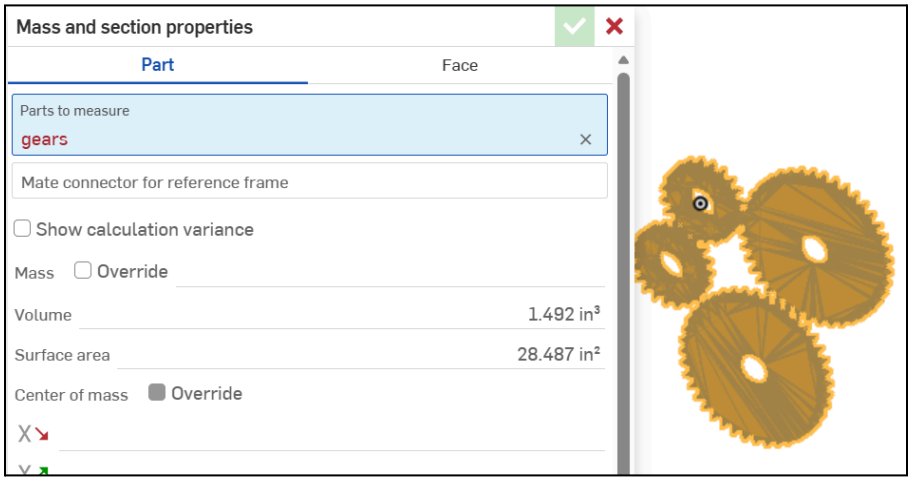
Left mount:



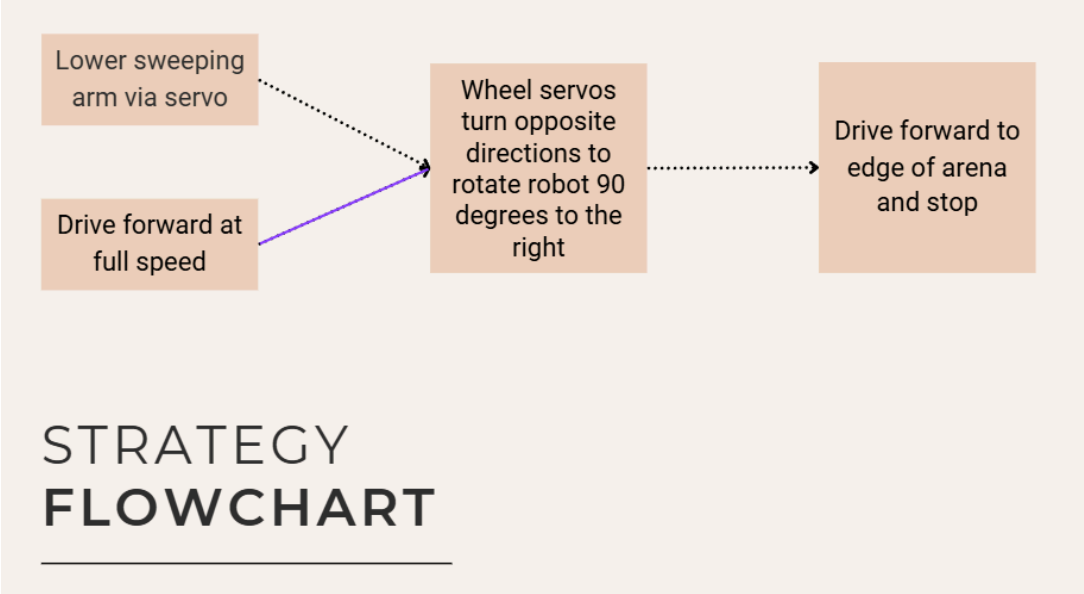
Right mount:



Gears (printed as 1 part with small tabs between gears to break apart into 4 gears):



8. Appendix D: Flowchart



9. Appendix E: Code: commented Arduino code. A clear understanding of each line of code should be demonstrated. References must be provided if used.

```
void initServos() //initialize servos
{
    DDRB = DDRB | 0b00000110; //sets PB1 and PB2 as outputs
    TCCR1A = 0b10100010; //configure Timer1 for PWM output on OC1A and OC1B
    ICR1 = 40000; //set PWM period for standard servo frequency
}

void servoL_3() { OCR1A = 4800; } //sets the servo in a position where the arm is down

void stop() { //sets all relevant motor outputs to 0
```

```
  PORTD = PORTD & 0b10000111;
}
void drive_backwards() { //both wheels reverse
  PORTD = PORTD & 0b10000111;
  PORTD = PORTD | 0b00110000;
}
void drive_forward() { //both wheels forward
  PORTD = PORTD & 0b10000111;
  PORTD = PORTD | 0b01001000;
}
void turn_left() { //left reverse, right forward
  PORTD = PORTD & 0b10000111;
  PORTD = PORTD | 0b00101000;
}
void turn_right() { //right reverse, left forward
  PORTD = PORTD & 0b10000111;
  PORTD = PORTD | 0b01010000;
}
void turn_left2() { //turning left with only 1 wheel (without left wheel reversing)
  PORTD = PORTD & 0b10000111;
  PORTD = PORTD | 0b00001000;
}
void turn_right2() { //turning right with only 1 wheel (without right wheel reversing)
  PORTD = PORTD & 0b10000111;
  PORTD = PORTD | 0b01000000;
}



void drive_forward_ms(int duration_ms) { //drive forward for a set time, then stop
  drive_forward();
  delay(duration_ms);
  stop();
}
void drive_backwards_ms(int duration_ms) { //drive backwards for a set time, then stop
  drive_backwards();
  delay(duration_ms);
  stop();
}
void turn_left_ms(int duration_ms) { //turn left for a set time, then stop (1 wheel only)
  turn_left2();
  delay(duration_ms);
  stop();
}
void turn_right_ms(int duration_ms) {  //turn right for a set time, then stop (1 wheel only)
  turn_right2();
  delay(duration_ms);
  stop();
}
```

```
void turn_right_ms_both(int duration_ms) {  //turn right for a set time, then stop (using 2
wheels)
  turn_right();
  delay(duration_ms);
  stop();
}



int main(void) {

  init(); //initializes functions like delay
  DDRD = DDRD | 0b01111000; //sets PD3-6 as outputs
  initServos(); //initializes servos

  drive_forward(); //start driving
  servoL_3(); //bring arm down

  drive_forward_ms(850); //now drive straight for 850ms

  for (int i = 1; i <= 45; i++){ //alternates between 2wheel and 1wheel turning to get a
smoother turn for 900ms
    turn_right_ms_both(10); //left wheel forward, right wheel reverse
    turn_right_ms(10); //only left wheel forward
  }

  drive_forward_ms(600); //go forward for 600ms

  for (int i = 1; i <= 35; i++){ //alternates between 2wheel and 1wheel turning to get a
smoother turn for 700ms
    turn_right_ms_both(10); //left wheel forward, right wheel reverse
    turn_right_ms(10); //only left wheel forward
  }

  drive_forward_ms(600); //go forward for 600ms
  stop(); //stops the robot at the side of the board ideally having collected >10 cubes

  return 0; //exit main function
}
```

References: chatgpt.com was used to understand how to initialize the servo and use PWM to control it.